

VadaChain — Full Build Specification for IBM Bob

Purpose of this document: This is a strict, unambiguous build spec. Do not deviate from names, file structure, function signatures, or logic described here unless a section explicitly says "your choice." Build in the phase order given — do not skip ahead or combine phases. Each phase must be independently testable before moving to the next.

0. PROJECT OVERVIEW (context, read once before starting)

What this is: A multi-agent AI system (built with LangGraph) where specialized AI agents plan, execute, and independently verify a piece of work (e.g., a coding task). Payment for completed work is held in a Solidity smart contract on a testnet (escrow) and is released **only** when an independent "Auditor" agent — using its own separate blockchain wallet — cryptographically submits a passing verdict. No single agent, human, or backend process can move the escrowed funds alone; only the contract's encoded rule can.

Core principle to preserve at every step: the smart contract must enforce that *only the Auditor's designated wallet address* can trigger fund release for a given task. This is the entire trust guarantee of the project — do not implement a shortcut where the backend directly transfers funds based on reading Auditor output in Python. The Auditor's LLM output must be turned into an actual signed transaction from the Auditor's own wallet.

Primary demo task type: an objectively verifiable coding task — "write a Python function `fib(n)` that returns the nth Fibonacci number, passing 5 hidden unit tests." This is the only task type required for the MVP demo. Research/writing task types (with RAG) are a Phase 5 stretch addition, not required for core functionality.

1. FULL PROJECT FILE STRUCTURE

Create this exact structure. Do not flatten it or rename folders.

```

vadachain/
├─ README.md
├─ .env.example
├─ .gitignore
├─ requirements.txt
|
├─ contracts/
|   ├─ VadaChainEscrow.sol
|   ├─ hardhat.config.js
|   ├─ package.json
|   └─ scripts/
|       └─ deploy.js
|
├─ backend/
|   ├─ main.py                # FastAPI entrypoint
|   ├─ config.py              # env vars, constants, model names
|   └─ state.py                # AgentState, TaskSpec, ExecutorProfile,
AuditVerdict (Pydantic/TypedDict)
|   └─ graph.py                # LangGraph graph assembly (nodes +
edges wired together)
|   |
|   └─ nodes/
|       ├─ __init__.py
|       ├─ requester_node.py
|       ├─ matcher_node.py
|       └─ executor_node.py    # router: dispatches to
code/research/writing sub-nodes
|   |
|   ├─ executor_code.py
|   ├─ executor_research.py    # Phase 5 only
|   ├─ executor_writing.py     # Phase 5 only
|   ├─ auditor_node.py
|   ├─ escalation_node.py
|   └─ reporter_node.py
|   |
|   └─ tools/
|       ├─ __init__.py
|       └─ code_sandbox.py     # sandboxed subprocess executor with
resource limits
|   |
|   ├─ web_search_tool.py      # Phase 5 only
|   ├─ vector_store.py         # Chroma client wrapper, two collections
|   └─ llm_client.py           # Groq primary + fallback provider logic
|   |
|   └─ chain/

```

```
| | | └─ __init__.py
| | | └─ contract_client.py      # web3.py wrapper: lockPayment,
submitVerdict, reclaimAfterTimeout, reputation reads
| | | └─ wallets.py              # wallet loading (Client, Auditor,
Executor N wallets) from env
| | |   └─ abi/
| | |       └─ VadaChainEscrow.json # compiled ABI, generated by hardhat,
copied here
| | |
| | └─ tests/
| |     └─ test_requester_node.py
| |     └─ test_matcher_node.py
| |     └─ test_executor_code.py
| |     └─ test_auditor_node.py
| |     └─ test_contract_client.py
| |     └─ test_graph_end_to_end.py
|
└─ frontend/
    └─ package.json
    └─ src/
        └─ App.jsx
        └─ components/
            └─ TaskSubmitForm.jsx
            └─ PipelineTimeline.jsx # shows
Requester→Matcher→Executor→Auditor→Escrow live
| | | | └─ AuditHistoryPanel.jsx # shows retry attempts + critiques
| | | |   └─ TransactionLink.jsx  # links out to Sepolia/Amoy explorer
| | | └─ api/
| | |     └─ client.js            # calls backend FastAPI endpoints
|
└─ docs/
    └─ architecture_diagram.png
    └─ demo_script.md
```

2. TECH STACK — PINNED, DO NOT SUBSTITUTE WITHOUT ASKING

Layer	Choice
Agent orchestration	LangGraph (Python)
LLM provider (primary)	Groq API, model <code>llama-3.3-70b-versatile</code>
LLM provider (fallback)	OpenRouter (free-tier model) — only invoked if Groq call fails or rate-limits
Vector store	Chroma, local persistent client (no cloud)
Backend framework	FastAPI
Blockchain library (Python side)	web3.py
Smart contract language	Solidity ^0.8.20
Contract dev framework	Hardhat
Testnet	Polygon Amoy (use this unless told otherwise)
RPC provider	Alchemy free tier
Frontend	React (Vite), plain CSS or Tailwind — no component library required
Hosting (backend)	Render free tier
Hosting (frontend)	Vercel free tier

3. PHASED BUILD PLAN

Build strictly in this order. Each phase ends with a stated "done when" checkpoint — do not proceed until that checkpoint is met.

PHASE 1 — State Schema + Happy-Path Graph (no blockchain, no RAG, mocked audit)

Goal: Get `Requester → Matcher (mocked) → Executor(code) → Auditor → Reporter` running end-to-end in-memory, no external dependencies except the LLM API.

Files to build: `backend/state.py`, `backend/nodes/requester_node.py`, `backend/nodes/matcher_node.py` (mocked — return a hardcoded list of 2 fake ExecutorProfiles, no vector store yet), `backend/nodes/executor_node.py`,

`backend/nodes/executor_code.py`, `backend/tools/code_sandbox.py`,
`backend/nodes/auditor_node.py` (real logic: actually run unit tests via `code_sandbox.py`, no
RAG yet), `backend/nodes/reporter_node.py`, `backend/graph.py`, `backend/main.py` (one
endpoint: `POST /run_task` that takes a raw request string and runs the graph to completion,
returns full final state as JSON).

Exact state schema (implement in `backend/state.py` exactly as follows):

```
python
```

```

from typing import TypedDict, Literal, Optional
from pydantic import BaseModel

class TaskSpec(BaseModel):
    task_id: str
    description: str
    acceptance_criteria: list[str]
    budget: float
    task_type: Literal["code", "research", "writing"]
    deadline_unix: int

class ExecutorProfile(BaseModel):
    agent_id: str
    wallet_address: str
    skills: list[str]
    past_success_rate: float
    tasks_completed: int

class AuditVerdict(BaseModel):
    passed: bool
    confidence: float
    critique: str
    retrieved_precedents: list[str]  # empty list until Phase 4 (RAG)

class AgentState(TypedDict):
    raw_request: str
    task_spec: Optional[TaskSpec]
    candidate_executors: list[ExecutorProfile]
    chosen_executor: Optional[str]
    executor_output: Optional[str]
    audit_history: list[AuditVerdict]
    retry_count: int
    max_retries: int
    escrow_tx_hash: Optional[str]
    reclaim_tx_hash: Optional[str]
    reputation_tx_hash: Optional[str]
    final_status: Optional[Literal["paid", "refunded", "escalated"]]
    escalation_reason: Optional[str]

```

Requester Agent — exact requirement: LLM call with structured output forcing a `TaskSpec`. System prompt must instruct: "Rewrite the user's request into a task with objective, testable acceptance criteria. For coding tasks, acceptance criteria must be phrased as things a test suite can check (e.g. 'function returns correct value for n=0 through

n=10'), never subjective language like 'good' or 'clean!'" Set (`max_retries = 2`) as a default in this node when creating state.

Matcher Agent (mocked in Phase 1): Return exactly 2 hardcoded (`ExecutorProfile`) entries with (`skills=["code_exec"]`), differing (`past_success_rate`). Pick the higher one. No LLM call needed yet in Phase 1 — real semantic matching + LLM tie-break comes in Phase 4.

Executor (code) — exact requirement: LLM generates a Python function based on (`task_spec.description`). Output goes into (`executor_output`) as a string containing the raw code. Must NOT execute anything itself — execution happens only inside (`code_sandbox.py`), called by the Auditor.

(`code_sandbox.py`) — exact requirement: A function (`run_tests(code: str, test_code: str, timeout_seconds: int = 5, memory_limit_mb: int = 256) -> dict`) that:

- Writes (`code`) to a temp file
- Runs (`test_code`) (pytest-style, provided by Auditor) against it in a subprocess
- Enforces the timeout via (`subprocess.run(..., timeout=timeout_seconds)`)
- Enforces memory limit using (`resource.setrlimit`) (Linux) in the subprocess's `preexec_fn`
- No network access inside the subprocess (do not import requests/httpx inside the sandboxed script; if feasible, run with (`--no-site`) or a restricted environment)
- Returns (`{"passed": bool, "output": str, "num_tests_passed": int, "num_tests_total": int}`)

Auditor Agent — exact requirement (Phase 1 version, code tasks only): For (`task_type == "code"`), the Auditor has a **hardcoded internal test suite** for the fib(n) demo task specifically (5 hidden unit tests covering n=0,1,2,10,20). It calls (`code_sandbox.run_tests(...)`) and builds an (`AuditVerdict`) from the real pass/fail result — (`passed = (num_tests_passed == num_tests_total)`), (`confidence = num_tests_passed / num_tests_total`), (`critique`) = a plain-language summary of which tests failed and why (derived from (`output`), not invented). Append this verdict to (`audit_history`).

Conditional edge — exact requirement: implement (`route_after_audit(state) -> Literal["retry", "release", "escalate"]`):

```
python
```

```
def route_after_audit(state):
    last = state["audit_history"][-1]
    if last.passed:
        return "release"
    elif state["retry_count"] < state["max_retries"]:
        return "retry"
    else:
        return "escalate"
```

On `"retry"`: increment `retry_count`, route back to `executor_node`, and inject the latest critique into the Executor's prompt context (pass `state["audit_history"][-1].critique` into the Executor node's system/user message explicitly — this is required, not optional, it is what makes this a Reflexion loop rather than a blind retry).

Reporter Agent — exact requirement: Produce a final plain-text summary string covering: task description, chosen executor, number of attempts, final verdict, and (in Phase 1) a placeholder `"[escrow not yet implemented]"` where the transaction hash will go later.

main.py — exact requirement: One FastAPI POST endpoint `/run_task` accepting `{"raw_request": str}`, invoking the compiled LangGraph graph, returning the full final `AgentState` as JSON.

Done when: Calling `POST /run_task` with `{"raw_request": "write a function that returns the nth fibonacci number"}` returns a final state where `audit_history` has at least one entry, `final_status` is set, and if the first attempt fails, you can observe a second attempt in `audit_history` with a different `executor_output` reflecting the injected critique.

PHASE 2 — Smart Contract + Wallets (no backend integration yet)

Goal: Deployable, tested Solidity contract on Polygon Amoy testnet, callable manually via Hardhat scripts, before any Python integration.

File: `contracts/VadaChainEscrow.sol` — exact required functions and state:

solidity


```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract VadaChainEscrow {
    struct Task {
        address client;
        address executor;
        address auditor;
        uint256 amount;
        uint256 deadline;
        bool resolved;
    }

    mapping(uint256 => Task) public tasks;
    mapping(address => uint256) public tasksCompleted;
    mapping(address => uint256) public tasksSucceeded;

    event PaymentLocked(uint256 indexed taskId, address client, address executor, uint256 amount);
    event VerdictSubmitted(uint256 indexed taskId, bool passed);
    event PaymentReleased(uint256 indexed taskId, address to, uint256 amount);
    event ReclaimedAfterTimeout(uint256 indexed taskId, address client, uint256 amount);

    function lockPayment(uint256 taskId, address executor, address auditor, uint256 amount) public {
        require(tasks[taskId].client == address(0), "Task already exists");
        require(msg.value > 0, "Must send payment");
        tasks[taskId] = Task(msg.sender, executor, auditor, msg.value, deadline, false);
        emit PaymentLocked(taskId, msg.sender, executor, auditor, msg.value, deadline);
    }

    function submitVerdict(uint256 taskId, bool passed) external {
        Task storage t = tasks[taskId];
        require(t.client != address(0), "Task does not exist");
        require(!t.resolved, "Already resolved");
        require(msg.sender == t.auditor, "Only assigned auditor");
        t.resolved = true;
        tasksCompleted[t.executor] += 1;
        emit VerdictSubmitted(taskId, passed);
        if (passed) {
            tasksSucceeded[t.executor] += 1;
            payable(t.executor).transfer(t.amount);
            emit PaymentReleased(taskId, t.executor, t.amount);
        } else {
            payable(t.client).transfer(t.amount);
        }
    }
}

```

```

        emit PaymentReleased(taskId, t.client, t.amount);
    }
}

function reclaimAfterTimeout(uint256 taskId) external {
    Task storage t = tasks[taskId];
    require(t.client != address(0), "Task does not exist");
    require(!t.resolved, "Already resolved");
    require(msg.sender == t.client, "Only client");
    require(block.timestamp > t.deadline, "Not yet expired");
    t.resolved = true;
    payable(t.client).transfer(t.amount);
    emit ReclaimedAfterTimeout(taskId, t.client, t.amount);
}

function getSuccessRate(address executor) external view returns (uint256) {
    if (tasksCompleted[executor] == 0) return 0;
    return (tasksSucceeded[executor] * 100) / tasksCompleted[executor];
}
}

```

Do not add weighted reputation, category tags, or decay logic — explicitly out of scope for this build.

File: `contracts/scripts/deploy.js` — Hardhat script that deploys the contract to Polygon Amoy using RPC URL and private key from `.env`, prints the deployed contract address to console, and writes it to `backend/chain/deployed_address.txt`.

Wallets required (document in `README.md` under a "Wallet Setup" section, and load addresses/keys via `.env`):

- `CLIENT_PRIVATE_KEY` / `CLIENT_ADDRESS`
- `AUDITOR_PRIVATE_KEY` / `AUDITOR_ADDRESS`
- `EXECUTOR_1_PRIVATE_KEY` / `EXECUTOR_1_ADDRESS`
- `EXECUTOR_2_PRIVATE_KEY` / `EXECUTOR_2_ADDRESS`
- `DEPLOYER_PRIVATE_KEY` (for deploying the contract itself)

Each must be funded with free Polygon Amoy test MATIC from a public faucet — document the faucet URL in `README.md`.

Done when: Contract deploys successfully to Amoy, and you can manually call `lockPayment`, then `submitVerdict` from the Auditor's wallet only (test that calling `submitVerdict` from a *different* wallet reverts with "Only assigned auditor" — this specific negative test is required, not optional, since it's the core trust guarantee).

PHASE 3 — Python ↔ Contract Integration

Goal: Wire the Phase 1 graph to actually call the Phase 2 contract at the right points.

File: `backend/chain/wallets.py` — loads all 5 wallets from `.env` into `web3.Account` objects.

File: `backend/chain/contract_client.py` — exact required functions:

- `lock_payment(task_id: int, executor_address: str, auditor_address: str, amount_wei: int, deadline_unix: int) -> str` — signs and sends with the Client wallet, returns tx hash
- `submit_verdict(task_id: int, passed: bool) -> str` — signs and sends **with the Auditor wallet specifically**, returns tx hash. This function must only ever use the Auditor's private key — never the Client's or a generic backend key.
- `reclaim_after_timeout(task_id: int) -> str` — signs with Client wallet
- `get_success_rate(executor_address: str) -> int` — read-only call, no gas

Graph changes required:

- New node `escrow_lock_node`, inserted between `matcher_node` and `executor_node`, calling `lock_payment(...)` and storing the result in `state["escrow_tx_hash"]`
- New node `escrow_release_node`, inserted after the `"release"` branch of `route_after_audit`, calling `submit_verdict(task_id, passed=True)`
- New node `escrow_refund_node`, inserted after the `"escalate"` branch (post human decision) or directly on final `"retry"` exhaustion depending on your escalation design — calling `submit_verdict(task_id, passed=False)`
- `matcher_node` must now call `get_success_rate(...)` for each candidate from the real contract instead of using the hardcoded `past_success_rate` field

Done when: Running `/run_task` end-to-end produces a real `escrow_tx_hash` and (on pass) a real payment transaction visible on the Amoy block explorer, and manually attempting to call `submit_verdict` from the backend using a non-Auditor key fails as expected (confirms you didn't accidentally bypass the wallet restriction).

PHASE 4 — RAG Layer (Matcher + Auditor)

Goal: Add the two genuinely load-bearing RAG uses. Do not add RAG anywhere else.

File: `backend/tools/vector_store.py` — Chroma persistent client with exactly two collections: `task_precedents` (past task descriptions + their final audit verdicts + critiques) and `source_documents` (for Phase 5 research/writing tasks only).

Matcher Agent update: Before picking an Executor, embed the current `task_spec.description`, retrieve top-3 most similar entries from `task_precedents`, and note which Executor(s) succeeded on those similar past tasks. Combine this with the on-chain

`get_success_rate` call from Phase 3. LLM call only used to break a tie between two close candidates — must not override a clear quantitative winner.

Auditor Agent update: Before finalizing a verdict, retrieve top-3 most similar past entries from `task_precedents` and include them in `retrieved_precedents` on the `AuditVerdict`. For code tasks this is supplementary context only (the real test-run result still determines `passed`) — do not let retrieved precedent override an objective test result.

After every completed task (in `reporter_node` or a new small `memory_write_node`): write the task description + final verdict + critique into `task_precedents` so future tasks benefit from it. This is what makes memory persist across runs, not just within one graph execution.

Done when: Running two similar coding tasks back-to-back shows the second task's Matcher/Auditor output referencing the first task's outcome in `retrieved_precedents`.

PHASE 5 — Stretch: Research/Writing Task Type + Escalation Node + Frontend

Only start this phase after Phases 1–4 are fully working and tested.

- `executor_research.py` / `executor_writing.py`: use `web_search_tool.py` or retrieval from `source_documents` collection to ground output; Auditor checks claims against the same retrieved sources
- `escalation_node.py`: on retries exhausted, expose `audit_history` + `escalation_reason` via a FastAPI endpoint for manual human pass/fail input (simple approve/reject, no auth needed for demo)
- Frontend: `PipelineTimeline.jsx` polls `/task_status/{task_id}` and shows each node's completion live; `TransactionLink.jsx` renders `https://amoy.polygonscan.com/tx/{tx_hash}` for every tx hash in state
- `llm_client.py`: implement Groq call first, catch rate-limit/error, fall back to OpenRouter free-tier model automatically — required before any live demo, not optional polish

Done when: A research-type task runs end-to-end with a real retrieved source document, and the frontend shows a live-updating timeline plus clickable Amoy explorer links for every transaction in a completed run.

4. NON-NEGOTIABLE CONSTRAINTS (repeat for emphasis)

1. `submit_verdict` must only ever be signed by the Auditor's wallet — never a shared/generic backend key.
2. No feature outside this document (weighted reputation, dispute windows, category tags, decay, multi-chain support) should be added without being asked for first.

3. All blockchain activity stays on testnet (Polygon Amoy). Never configure mainnet RPC or mainnet private keys anywhere in this project.
 4. Code execution for the Executor's output must always go through `code_sandbox.py` — never `eval`/`exec` directly in the main process.
 5. RAG is used only in Matcher and Auditor (and Executor for research/writing tasks in Phase 5) — do not add retrieval calls to escrow or reputation logic.
 6. Keep every phase's files separated exactly as listed in Section 1 — do not merge nodes into one large file "for convenience."
-

5. WHAT TO HAND BACK AFTER EACH PHASE

After completing each phase, provide: (a) a list of files created/modified, (b) the exact test/command used to verify the "done when" checkpoint, (c) its output/result, (d) any deviation from this spec and why, flagged explicitly rather than silently changed.